Lab 1: Logic Document

By: Ashritha Achutharaman
Due: March 18, 2026

Classes Implemented:

1. Student
2. Course
3. Professor
4. Grades
5. LoginUser
6. CheckMyGradeSystem

Methods Used:

- Hash tables
- Lists
- Dictionaries

Why these methods?

Hashing was used in the LoginUser class for password handling because passwords should be stored securely rather than in plain text. By hashing passwords, the system can verify a user during login without keeping the original password visible or directly accessible. This makes the login process safer and more appropriate for an authentication feature.

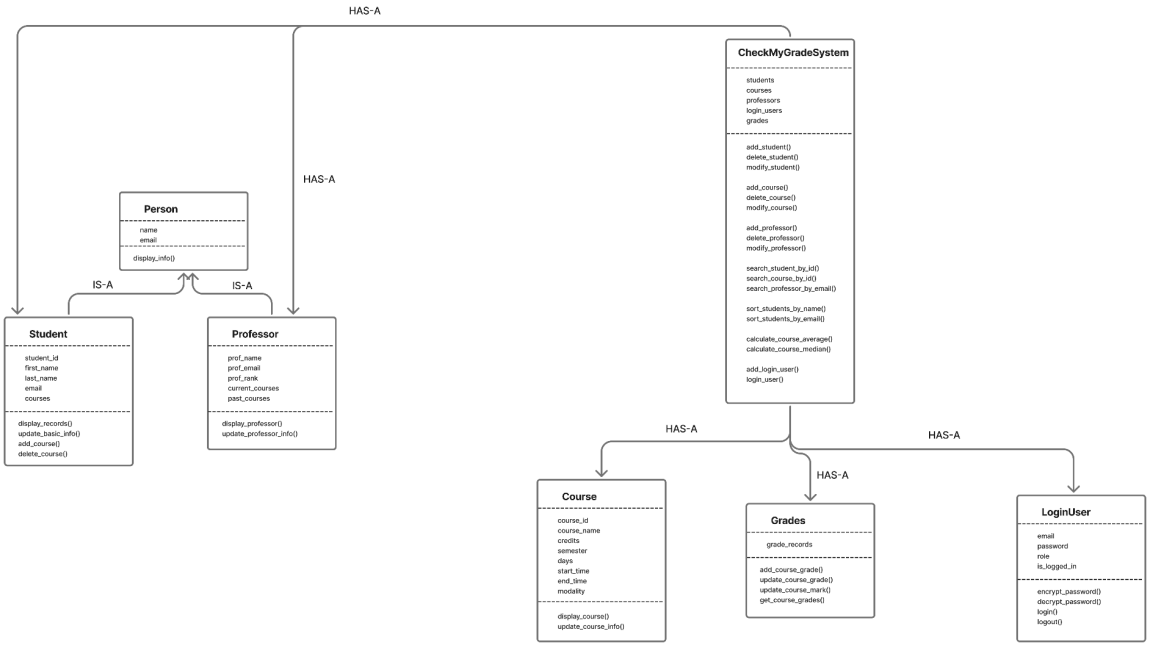
Hash tables were used throughout the program through Python dictionaries because the system needs fast access to records using unique identifiers. For example, students are searched by student_id, courses by course_id, professors by prof_email, and login records by email_id. Since these values act as unique keys, dictionaries were a practical choice because they allow records to be added, searched, updated, and deleted efficiently.

Lists were used for course collections such as a student's enrolled courses and a professor's current courses because these items are simple groups of related values that need to be stored together and displayed in order. A list was sufficient since these features mainly required storing multiple course IDs rather than constant key-based lookup.

Linked lists, stacks, and queues were not used because the system did not require those types of behavior. The program does not process data in sequential node form like a linked list, nor does it rely on last-in-first-out behavior like a stack or first-in-first-out behavior like a queue.

Instead, the main need was efficient storage, lookup, and updating of records, which is why hash tables and lists were the most appropriate choices.

Relationships:



Code Functionalities:

Student:

```python
class Student:
    def __init__(self, student_id, first_name, last_name, email, courses=None):
        self.student_id = student_id
        self.first_name = first_name
        self.last_name = last_name
        self.email = email
        self.courses = courses if courses is not None else []

    def add_course(self, course_id):
        if course_id in self.courses:
            print("Course already added to schedule.")
        else:
            self.courses.append(course_id)
            print("Course added successfully.")

    def delete_course(self, course_id):
        if course_id in self.courses:
            self.courses.remove(course_id)
            print("Course removed successfully.")
        else:
            print("Course not found.")

    def display_records(self):
        print(f"Student ID: {self.student_id}")
        print(f"First Name: {self.first_name}")
        print(f"Last Name: {self.last_name}")
        print(f"Email: {self.email}")

        if not self.courses:
            print("No courses enrolled.")
        else:
            print("Enrolled Courses:")
            for course_id in self.courses:
                print(f"  {course_id}")
```

The Student class represents a student in the system and stores their personal information along with the courses they are enrolled in. Each student object contains details such as the student ID, first name, last name, email address, and a list of courses.

This class allows the system to manage a student's course schedule. A student can add courses to their schedule, remove courses if they drop them, and view their current enrollment

information. The class also provides a way to display the student's basic information and the list of courses they are taking.

Course:

```python
class Course:
    def __init__(self, course_id, course_name, course_creds, term, days, start_time, end_time, modality):
        self.course_id = course_id
        self.course_name = course_name
        self.course_creds = course_creds
        self.term = term
        self.days = days
        self.start_time = start_time
        self.end_time = end_time
        self.modality = modality

    def display_course_details(self):
        print("COURSE DETAILS:")
        print(f"Course ID: {self.course_id}")
        print(f"Course Name: {self.course_name}")
        print(f"Number of Credits: {self.course_creds}")
        print(f"Term Offered: {self.term}")
        print(f"Days of Class: {self.days}")
        print(f"Start Time: {self.start_time}")
        print(f"End Time: {self.end_time}")
        print(f"Mode of Class: {self.modality}")
```

The Course class represents a course offered by the system and stores information about that course. Each course object contains details such as the course ID, course name, number of credits, term in which the course is offered, the days the class meets, the start and end times, and the class modality (for example, online or in-person).

This class allows the program to store and display course information in an organized way. It provides a method to display all the course details so that users can easily view the information about a specific course.

Professor:

```python
class Professor:
    def __init__(self, prof_name, prof_email, prof_rank, current_courses=None, past_courses=None):
        self.prof_name = prof_name
        self.prof_email = prof_email
        self.prof_rank = prof_rank
        self.current_courses = current_courses if current_courses is not None else []
        self.past_courses = past_courses if past_courses is not None else {}

    def prof_details(self):
        print(f"Professor Name: {self.prof_name}")
        print(f"Professor Email: {self.prof_email}")
        print(f"Professor Rank: {self.prof_rank}")

    def prof_course_details(self):
        print(f"Courses for Professor: {self.prof_name}")

        if not self.current_courses:
            print("Current Courses: None")
        else:
            print("Current Courses:")
            for course_id in self.current_courses:
                print(f"  {course_id}")

        if not self.past_courses:
            print("Previously Taught Courses: None")
        else:
            print("Past Courses:")
            for year in self.past_courses:
                print(f"  {year}:")
                for course_id in self.past_courses[year]:
                    print(f"    {course_id}")
```

The Professor class represents a professor in the system and stores information about them, including their name, email, rank, the courses they are currently teaching, and the courses they have taught in the past. Each professor object keeps both personal and teaching-related information organized in one place.

This class allows the program to display a professor's basic details as well as their teaching history. It can show the professor's name, contact information, and academic rank, along with a list of their current courses and previously taught courses. This makes it useful for keeping track of faculty records and course assignments.

Grades:

```python
class Grades:
    def __init__(self):
        self.grade_records = {}

    def add_student_record(self, student_id):
        if student_id not in self.grade_records:
            self.grade_records[student_id] = {}

    def add_course_grade(self, student_id, course_id, course_grade="Not Available Yet", marks=None):
        if student_id not in self.grade_records:
            self.grade_records[student_id] = {}

        self.grade_records[student_id][course_id] = {
            "grade": course_grade,
            "marks": marks if marks is not None else {}
        }

    def delete_course_grade(self, student_id, course_id):
        if student_id in self.grade_records and course_id in self.grade_records[student_id]:
            del self.grade_records[student_id][course_id]
            print("Grade record deleted successfully.")
        else:
            print("Course or student not found in records.")

    def update_course_grade(self, student_id, course_id, new_grade):
        if student_id in self.grade_records and course_id in self.grade_records[student_id]:
            self.grade_records[student_id][course_id]["grade"] = new_grade
            print("Grade updated successfully.")
        else:
            print("Course or student not found in records.")

    def update_course_mark(self, student_id, course_id, item, new_score):
        if student_id in self.grade_records and course_id in self.grade_records[student_id]:
            self.grade_records[student_id][course_id]["marks"][item] = new_score
            print("Marks have been updated successfully.")
        else:
            print("Course or student not found in records.")
```

```python
    def show_student_grades(self, student_id):
        if student_id not in self.grade_records or not self.grade_records[student_id]:
            print("No grade records available for this student.")
        else:
            print(f"Grade records for Student ID: {student_id}")
            for course_id in self.grade_records[student_id]:
                grade = self.grade_records[student_id][course_id]["grade"]
                print(f"{course_id}: {grade}")

    def show_student_marks(self, student_id):
        if student_id not in self.grade_records or not self.grade_records[student_id]:
            print("No marks available for this student.")
        else:
            print(f"Marks for Student ID: {student_id}")
            for course_id in self.grade_records[student_id]:
                print(f"Course ID: {course_id}")
                marks = self.grade_records[student_id][course_id]["marks"]

                if not marks:
                    print("No marks available.")
                else:
                    for item in marks:
                        print(f"  {item}: {marks[item]}")

    def show_student_report(self, student_id):
        if student_id not in self.grade_records or not self.grade_records[student_id]:
            print("No report available for this student.")
        else:
            print(f"FINAL REPORT FOR STUDENT ID: {student_id}")
            for course_id in self.grade_records[student_id]:
                print(f"\nCourse ID: {course_id}")
                print(f"Grade: {self.grade_records[student_id][course_id]['grade']}")
                print("Marks:")

                marks = self.grade_records[student_id][course_id]["marks"]
                if not marks:
                    print("No marks available yet.")
                else:
                    for item in marks:
```

```python
else:
    for item in marks:
        print(f"  {item}: {marks[item]}")
```

The Grades class is used to manage and store academic performance records for students. It keeps track of each student's courses, final grades, and individual marks for assignments, exams, or other graded items. This class serves as a gradebook system where student performance information can be organized in one place.

This class allows the program to create grade records for students, add course grades, update grades and marks, remove grade records for a course, and display different types of academic information. It can show a student's grades by course, display detailed marks for each course, and generate a full report that includes both final grades and individual scores.

Login:

```python
class LoginUser:
    def __init__(self, email_id, password, role, already_hashed=False):
        self.email_id = email_id
        if already_hashed:
            self.password = password
        else:
            self.password = self.hash_password(password)
        self.role = role
        self.is_logged_in = False

    def hash_password(self, password):
        return hashlib.sha256(password.encode()).hexdigest()

    def login(self, entered_password):
        entered_password_hash = self.hash_password(entered_password)

        if entered_password_hash == self.password:
            self.is_logged_in = True
            print("Login successful!")
        else:
            print("Invalid password.")

    def logout(self):
        if self.is_logged_in:
            self.is_logged_in = False
            print("Logout successful.")
        else:
            print("User is not logged in.")

    def change_password(self, old_password, new_password):
        old_password_hash = self.hash_password(old_password)

        if old_password_hash == self.password:
            self.password = self.hash_password(new_password)
            print("Password changed successfully!")
        else:
            print("Password is incorrect.")
```

The LoginUser class is responsible for managing user authentication within the system. It stores essential login information including the user's email ID, password, role, and login status. This class allows the system to verify user credentials, manage login sessions, and allow users to update their passwords.

For security purposes, the system uses hashing to store passwords instead of saving them as plain text. When a new user account is created, the password is converted into a hashed value using the SHA-256 hashing algorithm. The original password is not stored in the system.

During login, the password entered by the user is hashed using the same algorithm, and the resulting hash is compared with the stored hashed password. If the two hashes match, the user is successfully authenticated and logged into the system.

CheckMyGradeSystem:

```python
class CheckMyGradeSystem:
    def __init__(self):
        self.students = {}
        self.courses = {}
        self.professors = {}
        self.logins = {}
        self.grades = Grades()

    def add_student(self, student):
        if not student.student_id:
            print("Student ID cannot be null.")
            return
        if student.student_id in self.students:
            print("Student ID must be unique.")
            return
        self.students[student.student_id] = student
        print("Student added successfully.")

    def add_course(self, course):
        if not course.course_id:
            print("Course ID cannot be null.")
            return
        if course.course_id in self.courses:
            print("Course ID must be unique.")
            return
        self.courses[course.course_id] = course
        print("Course added successfully.")

    def add_professor(self, professor):
        if not professor.prof_email:
            print("Professor email/ID cannot be null.")
            return
        if professor.prof_email in self.professors:
            print("Professor ID must be unique.")
            return
        self.professors[professor.prof_email] = professor
        print("Professor added successfully.")
```

```python
def add_login_user(self, login_user):
    if login_user.email_id in self.logins:
        print("Login user already exists.")
        return

    self.logins[login_user.email_id] = {
        "password": login_user.password,
        "role": login_user.role
    }
    print("Login user added successfully.")
```

```python
def login_user(self, email_id, entered_password):
    if email_id in self.logins:
        stored_hashed_password = self.logins[email_id]["password"]
        role = self.logins[email_id]["role"]

        user = LoginUser(email_id, stored_hashed_password, role, already_hashed=True)
        user.login(entered_password)
        return user
    else:
        print("User not found.")
        return None
```

The CheckMyGradeSystem class serves as the central management component of the application. It is responsible for organizing and controlling the main records in the system, including students, courses, professors, login accounts, and grade information. Rather than having each part of the program operate independently, this class brings everything together into one unified system.

Its purpose is to allow new records to be added while making sure important identifying information is valid and unique. This helps maintain accurate and consistent data across the application. It also supports the login process by managing stored user credentials and verifying whether a user can successfully access the system.

Output:

```
Student added successfully.
Student added successfully.
Student added successfully.
Student added successfully.
Student added successfully.
Student added successfully.
Student added successfully.
Student added successfully.
Student added successfully.
Student added successfully.
Student added successfully.
Student added successfully.
Student added successfully.
Student added successfully.
Student added successfully.
Student added successfully.
Student added successfully.
Student added successfully.
Student added successfully.
Student added successfully.
Student added successfully.
Student added successfully.
Student added successfully.
Student added successfully.
Student added successfully.
Student added successfully.
Student added successfully.
Student added successfully.
Student added successfully.
Student added successfully.
Student added successfully.
Student added successfully.
Student added successfully.
Student added successfully.
Student added successfully.
Student added successfully.
Student added successfully.
Student added successfully.
Student added successfully.
Student added successfully.
Student added successfully.
Student added successfully.
Student added successfully.
Student added successfully.
Student added successfully.
Student added successfully.
Student added successfully.
Student added successfully.
Student added successfully.
Student added successfully.
Student added successfully.
Student added successfully.
Student added successfully.
Student added successfully.
Student added successfully.
Student added successfully.
Student added successfully.
```

```
.Login user added successfully.
Invalid password.
.Login user added successfully.
Login successful!
.Course added successfully.
Course record updated successfully.
.Professor added successfully.
Professor record updated successfully.
.Student added successfully.
Student record updated successfully.
..Course added successfully.
Course found.
COURSE DETAILS:
Course ID: C101
Course Name: Python Programming
Number of Credits: 3
Term Offered: Spring
Days of Class: MW
Start Time: 10:00
End Time: 11:15
Mode of Class: In Person
Search time: 0.000008 seconds
.Professor added successfully.
Professor found.
Professor Name: Dr. Rao
Professor Email: rao@university.edu
Professor Rank: Associate Professor
Search time: 0.000003 seconds
.Student added successfully.
Student found.
Student ID: S101
First Name: Ashritha
Last Name: Achutharaman
Email: ash@example.com
Enrolled Courses:
  C101
```

```
Search time: 0.000006 seconds
.Student not found.
Search time: 0.000002 seconds
.Student added successfully.
Student added successfully.
Sorting time: 0.000002 seconds
.Student added successfully.
Student added successfully.
Sorting time: 0.000002 seconds
.
----------------------------------------------------------------------
Ran 23 tests in 0.002s

OK
```